

FieldMeta.jl Implementation Notes

1 What Does Macro Expansion Generate?

The fastest way to understand this package is to look at what it ultimately produces.

```

1 @metadata bounds nothing Any
2 @metadata units "-" String
3
4 @bounds @units struct Muskingum
5     x::Float64 | (0.01, 0.5) | "m/s"
6     dt::Float64
7 end

```

Is equivalent to:

```

1 struct Muskingum           # pipe stripped clean
2     x::Float64
3     dt::Float64
4 end
5
6 # Each (type, field, key) → specialized method returning a constant
7 @inline _meta(::Type{<:Muskingum}, ::Val{:x}, ::Val{:bounds}) = (0.01, 0.5)
8 @inline _meta(::Type{<:Muskingum}, ::Val{:x}, ::Val{:units})  = "m/s"
9
10 # Per-key default fallback generated by @metadata
11 @inline _meta(::Type, ::Val, ::Val{:bounds}) = nothing
12 @inline _meta(::Type, ::Val, ::Val{:units})  = "-"

```

When the user calls `bounds(model, :x)`, internally it dispatches to `_meta(Muskingum, Val{:x}(), Val{:bounds}())`. Three-level priority: **specialized method** > **per-key default** > **MethodError (undeclared key)**. The hot path is pure method dispatch — no Dict lookups, fully type-stable.

2 Pipe Chain and Stackable Macros

`|` is a left-associative operator in Julia:

```

1 x::T | v1 | v2 | v3  ≡  ((x::T | v1) | v2) | v3

```

The leftmost value `v1` lives in the deepest `:|` node. Julia expands macros from outer to inner, so:

- `@bounds` (outermost) strips `v1`, passes `x::T | v2 | v3` to the next macro.
- `@units` strips `v2`, passes `x::T | v3`.
- `@description` strips `v3`, leaving the clean `x::T`.

The core implementation is `_strip_leftmost!(args, i)` (7 lines), which recursively locates the deepest `:|` node, replaces it in-place, and returns the stripped value.

3 When Does Type Checking Happen?

Phase	Action
Macro expansion	Read constraint <code>c</code> from <code>REGISTRY</code> ; error immediately if key undeclared
Struct definition	<code>let v = \$val; v isa c _typeerror(...)</code> runs once
Access (hot path)	Specialized method returns <code>\$val</code> directly — zero overhead

`REGISTRY` is written only when `@metadata` runs and read only during macro expansion; it is never touched at runtime.

4 Internal Function Call Chain

When `@bounds struct ... end` expands, the call structure is:

```

1  macro bounds(ex)
2    _stack(ex, :bounds, src)
3    _process(ex, src, emit_fn)
4      1. _find_struct(ex)           # locate :struct, pierce @with_kw
        macrocalls
5      2. for each field line:
6          a. _meta_slot(block, i)   # has pipe? -> (args, idx, fname)
7          b. _strip_leftmost!(args, idx) # strip leftmost | value in-place,
        return val
8          c. _emit!(methods, T, fname, key, val)
9              - REGISTRY[key]       # read constraint c (macro-expansion time,
        once)
10         - push! generated block:
11             let v=val; v isa c || _typeerror(...) end # runs at
        definition time
12             @inline _meta(::Type{<:T}, ::Val{f}, ::Val{k}) = val
13      3. return Expr(:block, esc(ex), methods...)
```

Step 2 is three sequential sub-steps ($a \rightarrow b \rightarrow c$), not nested calls. `ex` is mutated in-place during step 2 (pipe stripped); by step 3 it is already a clean struct expression.

Expansion Order with Multiple Stacked Macros

Stacking does not rely on a loop in the code — it is driven by Julia's own macro expansion mechanism. Expansion (compile time) and execution (runtime) are two distinct phases:

Expansion phase (outer $\rightarrow$ inner, compile time):

```

1  @bounds @units struct ... x | v1 | v2 end
2  @bounds expands -> block { esc(@units struct...x|v2 end), # ex first
3                      @inline _meta(...bounds...) = v1 }
4  @units expands -> block { esc(struct...x end),           # ex first
5                      @inline _meta(...units...) = v2 }
```

Execution phase (sequential, runtime):

```

1  1. struct ... x end                                <- struct defined first
```

```

2  2. @inline _meta(...units...) = v2      <- method references T, T already
   exists ✓
3  3. @inline _meta(...bounds...) = v1

```

_process always places `esc(ex)` before methods in the returned block, guaranteeing the struct is defined before any `_meta` method that references it.

5 Debug Entry Points

```

1  import FieldMeta: _ispipe, _find_struct, _fieldname, _strip_leftmost!,
   _meta_slot
2
3  # Feed quoted expressions directly to internal helpers
4  holder = Any[: (x::Int | (0, 1) | "m")]
5  _strip_leftmost!(holder, 1)    # => :((0, 1))
6
7  # Inspect the full expansion
8  @macroexpand @bounds @units struct Foo; x::Int | 1 | "u"; end

```

Every helper's behaviour is covered by runnable examples in `test/test-internals.jl`.